Relatório do Trabalho: Sistema de Locadora de Filmes

Gustavo Antonio de Sousa, Luan Cristan da Silva Rufino, Rian de Oliveira Leal

[1]

[2]Departamento de Sistemas de Informação
Universidade Federal do Piauí, Picos, Piauí, Brasil

gustavo.antonio@ufpi.edu.br, luan.rufino@ufpi.edu.br, rian.leal@ufpi.edu.br

Abstract. *This document presents the complete report on the development and evolution of a movie rental system. The group built the project using the Python programming language and the PySide6 library, dividing the development process into distinct stages. In the first phase, the entire graphical user interface was structured, incorporating signals, slots, events, various widgets, layouts, menus, and secondary windows operating in memory. In the final stage, the system was expanded to include permanent local storage via files (JSON, CSV, and TXT) and parallel execution (multithreading), enabling batch data importation and the creation of automatic backups in the background without freezing the graphical user interface.*

Resumo. *Este documento apresenta o relatório completo do desenvolvimento e evolução do sistema de Locadora de Filmes. O grupo construiu o projeto usando a linguagem Python e a biblioteca PySide6, dividindo o desenvolvimento em etapas. Na primeira fase, foi estruturada toda a interface gráfica com sinais, slots, eventos, múltiplos widgets, layouts, menus e janelas secundárias operando em memória. Na etapa final, o sistema foi expandido para integrar armazenamento local permanente em arquivos (JSON, CSV e TXT) e execução em paralelo (multithreading), permitindo a importação de dados em lote e a geração de backups automáticos em segundo plano sem congelar a interface gráfica do usuário.*

1. Informações Gerais e Descrição do Projeto

O projeto consiste em uma interface gráfica criada para gerenciar um catálogo de obras de uma locadora de filmes. Desde a primeira etapa do trabalho, foram criadas funcionalidades operacionais completas para listar as obras em uma tabela interativa, filtrar registros, cadastrar novos títulos através de um formulário dedicado, alugar, devolver e excluir filmes do catálogo. A interface gráfica foi construída inteiramente com a biblioteca PySide6, respeitando o tema central proposto na disciplina, e o código-fonte foi organizado desde o princípio em arquivos e módulos separados para facilitar a leitura e a manutenção do sistema.

Na segunda etapa e entrega final, aproveitando toda a base construída inicialmente, o sistema evoluiu de uma lista temporária em memória para uma aplicação com persistência de dados real em disco utilizando a biblioteca `json`. Além disso, foram adicionadas funcionalidades de manipulação de arquivos externos, permitindo a exportação

de todo o catálogo para o formato CSV e a importação em lote de grandes quantidades de filmes através de arquivos nos formatos TXT e CSV.

Para solucionar problema de travamento da interface gráfica durante operações demoradas de entrada e saída (I/O) de arquivos extensos, o projeto foi aprimorado com rotinas de processamento em paralelo (*Threads*). Dessa forma, a aplicação passou a carregar arquivos em segundo plano emitindo o progresso em tempo real na tela, ao mesmo tempo em que realiza cópias de segurança (*backups*) automáticas e silenciosas do banco de dados.

2. Como os Requisitos Foram Atendidos

Abaixo, é explicado e justificado como cada um dos requisitos técnicos e conceituais exigidos ao longo de toda a disciplina (Aulas 01 a 08) foi implementado na prática dentro do código.

2.1. Sinais e Slots

Conforme os conceitos introduzidos na Aula 01, o sistema possui diversas conexões nativas de sinais e slots, como as respostas aos cliques dos botões da interface (`clicked`), alterações de texto nos filtros (`textChanged`) e mudança de valor na barra deslizante de avaliação (`valueChanged`). Além dos sinais nativos, na primeira etapa foi criado um sinal customizado chamado `statusModificado = Signal(int, bool)` na janela de detalhes do filme. Esse sinal envia o identificador e o novo estado do filme para avisar a janela principal, forçando a atualização imediata da tabela visual logo após o usuário registrar uma locação ou devolução através da janela secundária.

Na segunda etapa, para integrar as novas rotinas de processamento em paralelo (Aula 08), foram criadas as classes `SinaisImportacao` e `SinaisBackup` herdando de `QObject`. O sinal `progresso = Signal(int)`, por exemplo, é emitido a cada linha processada na leitura do arquivo em lote e recebido por um slot na interface principal para preencher a `QProgressBar` de 0 a 100% em tempo real. Isso garante uma comunicação *thread-safe* (segura entre threads) entre o processamento de *background* e a interface gráfica.

2.2. Eventos

Baseando-se nos conceitos da Aula 02, foram reescritos eventos virtuais nativos da classe mãe na janela principal para melhorar a navegação e a segurança do sistema. Foi implementado evento de teclado (`keyPressEvent`) para que o programa abra a tela de detalhes do filme automaticamente quando o usuário seleciona uma linha da tabela e aperta a tecla *Enter* (`Key_Return` ou `Key_Enter`).

Também sobrescrevemos o evento de fechamento da janela (`closeEvent`). Inicialmente, ele foi implementado para exibir uma caixa de pergunta (`QMessageBox`) antes de encerrar o programa de forma definitiva e garantir o fechamento da janela secundária de detalhes caso estivesse aberta. Na entrega final, esse mesmo evento foi aprimorado para também desativar uma variável interna de controle (`self._backup_ativo = False`) assim que o usuário confirma a saída. Isso sinaliza a interrupção limpa do laço de repetição (*while*) da thread nativa de *backup*, prevenindo que processos órfãos continuem rodando na memória do sistema operacional (conceito abordado na Aula 08).

2.3. Layouts

A organização visual das telas foi construída combinando os gerenciadores de geometria estudados na Aula 02. Na área central da janela principal, foram empilhadas as páginas com o gerenciador `QStackedLayout`. Para alinhar os componentes da tela de catálogo (filtros superiores, tabela central, barra de disponibilidade e linha de botões inferiores), foi escolhido o `QVBoxLayout` para criar o empilhamento vertical e múltiplos `QHBoxLayout` aninhados para estruturar as linhas horizontais. Já no formulário de cadastro de novos filmes, foi utilizada uma grade invisível (`QGridLayout`) combinada com layouts horizontais para manter os rótulos de texto e os respectivos campos de entrada perfeitamente alinhados lado a lado.

2.4. Barras de Ferramentas e Menu

Apoiando-se nos conceitos da Aula 03, a tela principal exibe uma barra de menu (`QMenuBar`) no topo que abriga o menu de Ajuda e os créditos do sistema (”Sobre o Sistema”). Logo abaixo do menu, foi implementado uma barra de ferramentas fixa (`QToolBar`). Nela, inserimos atalhos para as ações principais do programa: cadastrar novo filme (`Ctrl+N`), ver detalhes (`Ctrl+D`), remover filme (`Delete`) e alternar visualização (`Ctrl+1`). Todas elas foram instanciadas através da classe abstrata `QAction`. Também foi adicionado um campo de texto (`QLineEdit`) diretamente dentro da barra de ferramentas para permitir ao usuário realizar uma busca rápida instantânea por título.

Na etapa final do projeto, o grupo realizou uma mudança visual na interface: removemos ações redundantes de importação e exportação que haviam sido testadas na `QToolBar`. Essa decisão manteve o topo do software limpo com as funções essenciais de navegação e registro, enquanto as operações de manipulação de arquivos em lote e exportação CSV ficaram concentradas de forma intuitiva nos botões dedicados na parte inferior da janela.

2.5. Dialogs e Alertas

Seguindo o conteúdo da Aula 04, o projeto utiliza extensamente alertas nativos (`QMessageBox`) para avisar o usuário sobre regras de preenchimento (como impedir o cadastro de um filme com título em branco), informar operações concluídas com sucesso e exibir perguntas de confirmação antes de excluir registros ou fechar o aplicativo. Além dos alertas básicos, construímos uma janela de diálogo dedicada herdando de `QDialog` (`DialogoCadastroFilme`) exclusiva para o cadastro de novos filmes, operando de forma modal (bloqueante) sobre a tela principal até que o usuário confirme os dados ou cancele a operação.

Na segunda etapa, foi integrado o diálogo nativo `QFileDialog` para permitir ao usuário escolher os caminhos de salvamento (na exportação CSV) e de abertura de arquivos (na importação TXT/CSV). Além disso, foi aprimorado o alerta emitido pelo `QMessageBox` ao término da importação em lote: caso nenhum filme seja inserido, o alerta informa expressamente que o formato é inválido *ou que os filmes já existem no catálogo*, refletindo a regra de negócio que impede o cadastro de títulos duplicados.

2.6. Pelo Menos Uma Janela Adicional

Para cumprir a exigência de ter uma tela extra independente (Aula 04), o sistema conta desde a primeira versão com uma janela secundária nomeada de `JanelaDetalhesFilme`. Ela herda diretamente da classe `QWidget` e exibe a ficha técnica completa do filme selecionado, agrupada dentro de um `QGroupBox`. Por ser instanciada sem um widget "pai"fixo, ela funciona de forma totalmente solta (não-modal), permitindo ao usuário interagir livremente com a janela principal ao fundo enquanto os detalhes continuam visíveis na frente, sincronizando alterações de locação em tempo real entre ambas as telas.

2.7. Persistência de Arquivos e Manipulação de Dados

Empregando os conhecimentos das Aulas 05, 06 e 07 na segunda fase do trabalho, o repositório deixou de armazenar os dados apenas em memória volátil e passou a gerenciar arquivos reais no disco:

- **Armazenamento em JSON:** Foi utilizada a biblioteca nativa `json` (`json.load` e `json.dump`) com o parâmetro `ensure_ascii=False` (Aula 06) para carregar e salvar o catálogo automaticamente no arquivo `dados.json`, preservando acentos e caracteres especiais em todas as operações de cadastro, remoção e locação.
- **Exportação e Importação em CSV e TXT:** Foi aplicado o módulo `csv` (`csv.reader` e `csv.writer`) utilizando a codificação `utf-8-sig`. Essa codificação foi escolhida porque adiciona o marcador BOM ao arquivo, fazendo com que softwares externos de planilha (como o Excel) reconheçam a acentuação em português sem desconfigurar os caracteres. O leitor de importação detecta dinamicamente delimitadores por vírgula (`,`) ou ponto e vírgula (`;`), limpa os campos e verifica se o filme já consta no acervo (comparando os atributos sem considerar o ID e o status de disponibilidade) para evitar duplicações.
- **Gerenciamento de Diretórios e Cópias:** Foram utilizados os módulos `os` (`os.path.exists` e `os.makedirs`, vistos na Aula 05) e `shutil` (`shutil.copy2`) para criar automaticamente a pasta `backups/` e gravar cópias de segurança do arquivo `dados.json` identificadas com a data e hora exatas (*timestamp*).

2.8. Multithreading e Paralelismo

Para solucionar o problema da interface gráfica travada durante a manipulação de arquivos (Aula 8), o projeto implementou na prática as duas abordagens de execução concorrente apresentadas em:

- **Threads no PySide6 (`QRunnable` e `QThreadPool`):** A importação de arquivos em lote é encapsulada na classe `TrabalhadorImportacao`, que herda de `QRunnable`, sendo gerenciada e escalonada por uma instância de `QThreadPool`. Isso move a leitura das linhas e a pausa visual (`time.sleep(0.3)`) para fora da thread da GUI, mantendo a janela principal responsiva enquanto a barra de progresso avança linha por linha.
- **Threads Nativas do Python (`threading.Thread`):** A rotina de backup periódico em segundo plano utiliza o módulo nativo `threading.Thread` configurado com o parâmetro `daemon=True`. Ela executa um laço contínuo nos

bastidores que gera uma cópia de segurança a cada 60 segundos e notifica a barra de status; por ser uma thread *daemon*, o sistema garante que ela não impeça o encerramento imediato da aplicação quando a janela principal é fechada.

2.9. Documentação de Todos os Widgets Criados

Para cumprir a exigência documental do projeto, está apresentado abaixo a lista detalhada contendo os nomes das variáveis instanciadas, o tipo de cada componente e a respectiva função na interface em todas as janelas do sistema:

Na Janela Principal (`janela_principal.py`):

- `campo_busca_rapida` (QLineEdit): recebe o texto para busca rápida diretamente na barra de ferramentas.
- `status_bar` (QStatusBar): exibe mensagens temporárias de operação, o percentual de importação e confirmações silenciosas de cópias de *backup* no rodapé.
- `layout_pilha` (QStackedLayout): gerencia a estrutura de páginas centrais da janela.
- `campo_filtro_titulo` (QLineEdit): recebe o texto para filtrar os filmes pelo título no painel superior.
- `combo_filtro_genero` (QComboBox): seleciona o gênero para filtrar a listagem da tabela.
- `tabela_filmes` (QTableWidget): exibe o catálogo completo de obras em formato tabular de 8 colunas.
- `label_progresso` (QLabel): rótulo dinâmico que alterna seu texto para indicar se a barra abaixo exibe a disponibilidade do acervo ou o progresso da importação em lote.
- `barra_progresso` (QProgressBar): indica visualmente a porcentagem de filmes disponíveis para locação e, durante a execução da thread de importação, mostra o progresso de leitura do arquivo de 0 a 100%.
- `btn_locar` (QPushButton): aciona a locação ou a devolução do item selecionado na tabela.
- `btn_exportar` (QPushButton): abre o diálogo de salvamento e aciona a exportação completa do catálogo para um arquivo `.csv`.
- `btn_importar` (QPushButton): abre o diálogo de seleção de arquivos (`.txt` ou `.csv`) e inicia a thread de importação em lote.

No Diálogo de Cadastro (`dialogo_cadastro.py`):

- `campo_titulo` (QLineEdit): captura o título do novo filme.
- `combo_genero` (QComboBox): lista os gêneros disponíveis para escolha.
- `spin_ano` (QSpinBox): captura o ano de lançamento variando de 1900 a 2030.
- `spin_preco` (QDoubleSpinBox): captura o preço da diária do filme formatado com prefixo monetário.
- `radio_dvd`, `radio_bluray`, `radio_digital` (QRadioButton): definem o tipo de mídia de forma mutuamente exclusiva através do agrupador `grupo_midia` (`QButtonGroup`).
- `check_disponivel` (QCheckBox): marca o status inicial de disponibilidade para locação imediata.
- `slider_avaliacao` (QSlider): barra deslizante horizontal (0 a 10) para atribuir uma nota inicial.

- `label_valor_slider` (QLabel): exibe o valor numérico exato do slider na tela em tempo real.
- `texto_sinopse` (QTextEdit): captura a descrição longa ou sinopse da obra.
- `caixa_botoes` (QDialogButtonBox): agrupa os botões padrão de confirmação (`Ok`) e cancelamento (`Cancel`).

Na Janela de Detalhes (`janela_detalhes.py`):

- `grupo_info` (QGroupBox): agrupa visualmente os rótulos da ficha técnica do filme.
- `label_titulo`, `label_genero`, `label_ano`, `label_preco`, `label_midia`, `label_status`, `label_avaliacao`, `label_sinopse` (QLabel): conjunto de rótulos dedicados para exibir os atributos do filme em modo de leitura.
- `botao_alternar_status` (QPushButton): botão dedicado para realizar a locação ou devolução diretamente pela janela secundária.
- `botao_fechar` (QPushButton): botão simples para ocultar (`hide`) a janela secundária.

2.10. Documentação das Classes, Funções e Métodos Criados

Para atender integralmente às especificações de documentação da arquitetura do software, é listada abaixo todas as classes, funções e métodos desenvolvidos nos módulos do projeto:

No módulo Principal (`main.py`):

- `main()`: Função de entrada que instancia a `QApplication`, exibe a `JanelaPrincipal` e inicia o *loop* de eventos com `app.exec()`.

No módulo de Dados (`dados/repositorio.py`):

- `RepositorioFilmes`: Classe que gerencia a persistência e as operações de negócio do catálogo.
- `__init__()`, `_carregar_dados()` e `_salvar_dados()`: Inicializam a lista interna, carregam os registros existentes do arquivo `dados.json` (ou criam a carga inicial padrão) e gravam alterações em disco.
- `listar_todos()` e `buscar_por_id(filme_id)`: Métodos de consulta que retornam a lista completa de obras ou um dicionário específico pelo seu identificador.
- `adicionar(...)`, `remover_por_id(filme_id)` e `alterar_status(filme_id, disponivel)`: Métodos de inserção, exclusão e atualização que modificam o estado do acervo e salvam imediatamente no JSON.
- `exportar_csv(caminho)`: Grava todos os filmes cadastrados em formato tabular CSV com cabeçalho padronizado.
- `importar_lote(caminho, callback_progresso)`: Lê arquivos TXT ou CSV, valida as 8 colunas de dados, previne o cadastro de títulos duplicados (ignorando ID e disponibilidade), salva os novos itens e emite o percentual concluído via *callback*.
- `fazer_backup()`: Cria a pasta `backups/` e gera uma cópia datada do arquivo `dados.json`.

No módulo de Janelas (`janelas/janela_principal.py`):

- `SinaisImportacao(QObject)` e `SinaisBackup(QObject)`: Classes auxiliares que definem os sinais customizados (`concluido`, `progresso`, `erro` e `backup_concluido`) para comunicação segura entre as threads secundárias e a GUI.
- `TrabalhadorImportacao(QRunnable)`: Classe *worker* cujo método `run()` executa `importar_lote` em uma thread gerenciada pelo `QThreadPool`.
- `JanelaPrincipal(QMainWindow)`: Classe principal que integra a interface gráfica, o repositório, a janela secundária e os gerenciadores de threads.
- `_criar_acoes()`, `_criar_menu_ajuda()`, `_criar_barra_ferramentas()`, `_criar_barra_status()` e `_criar_interface_central()`: Métodos construtores que estruturam os menus, a toolbar, os filtros, a tabela e os botões de ação.
- `_atualizar_tabela()` e `_obter_filme_selecionado()`: Aplicam os filtros ativos na listagem visual, recalculam a taxa de disponibilidade na barra de progresso e recuperam o registro selecionado pelo usuário.
- `_abrir_dialogo_cadastro()`, `_abrir_janela_detalhes()`, `_alternar_locacao_selecionado()`, `_remover_filme_selecionado()` e `_mostrar_sobre()`: Slots conectados às ações de gerenciamento do catálogo e exibição de diálogos.
- `_exportar_catalogo_csv()` e `_importar_catalogo_lote()`: Slots que abrem o seletor de arquivos (`QFileDialog`) e acionam a exportação direta ou a thread de importação em lote.
- `_slot_progresso_importacao()`, `_slot_importacao_concluida()` e `_slot_erro_importacao()`: Slots que recebem os sinais da thread de importação para atualizar a `QProgressBar` e exibir alertas de conclusão ou falha.
- `_slot_busca_rapida()` e `_slot_status_modificado_externamente()`: Sincronizam a busca da toolbar com o filtro principal e atualizam a tabela quando um filme é locado/devolvido na janela secundária.
- `_rotina_backup_automatico()` e `_slot_backup_concluido()`: Método executado em loop pela *daemon thread* nativa do Python e slot receptor que notifica o salvamento na barra de status.
- `closeEvent(event)` e `keyPressEvent(event)`: Eventos virtuais sobrescritos para encerramento seguro e atalho de teclado com a tecla *Enter*.

No módulo de Componentes (`componentes/`):

- `DialogoCadastroFilme(QDialog)`: Classe do formulário modal; possui os métodos `_atualizar_label_avaliacao(valor)` (sincroniza o rótulo com o slider), `_validar_e_aceitar()` (valida o preenchimento do título) e `obter_dados()` (retorna o dicionário com os dados preenchidos).
- `JanelaDetalhesFilme(QWidget)`: Classe da janela secundária independente; possui os métodos `exibir_filme(filme)` (preenche os rótulos com os dados da obra e traz a janela para a frente) e `_alternar_locacao()` (inverte a disponibilidade e emite o sinal `statusModificado`).

3. Uso de Inteligência Artificial

A Inteligência Artificial atuou de forma controlada durante todo o desenvolvimento do projeto, servindo somente como ferramenta de auxílio, consulta conceitual e procura de erros, em conformidade com as regras impostas. Nenhuma interface pronta ou código complexo partiu da ferramenta. Os modelos utilizados foram o Gemini e o Claude.

Tanto na primeira quanto na segunda etapa do trabalho, a IA foi utilizada para pesquisar propriedades específicas do PySide6 que complementavam as notas de aula, formatar dados de teste e auxiliar na identificação de erros. Após compreender a lógica explicada pela ferramenta, o grupo implementou e adaptou todo o código manualmente nos seguintes cenários:

- **QButtonGroup**: Na primeira etapa, foi pesquisado como agrupar logicamente os botões de rádio (`QRadioButton`) do tipo de mídia no formulário de cadastro, garantindo a seleção mutuamente exclusiva de forma nativa.
- **QTableWidgetItem**: Foi consultado como instanciar e inserir os textos individuais dentro das células de cada linha e coluna da tabela do catálogo.
- **QHeaderView**: Foi usada a ferramenta para entender como aplicar o modo de redimensionamento `Stretch`, forçando a coluna de título a expandir e ocupar todo o espaço horizontal restante da tabela.
- **setSelectionBehavior e setEditTriggers**: Foram pesquisaados esses métodos para configurar a tabela de modo a selecionar a linha inteira ao invés de apenas uma célula isolada, além de bloquear a edição direta dos textos na grade por parte do usuário.
- **setStyleSheet e setWordWrap**: Foi consultado como aplicar formatações visuais básicas nas fontes (tamanho e negrito no título) e como fazer o texto longo da sinopse sofrer quebra de linha automática na janela secundária de detalhes.
- **raise_() e activateWindow()**: Foi solicitada a explicação desses comandos para garantir que a janela secundária de detalhes sempre apareça no topo da tela, por cima da janela principal, ao ser acionada.
- **Compreensão do Ecossistema Qt vs Python Nativo**: Na segunda etapa, foi buscado exemplos para consolidar a integração de sinais customizados dentro do `QRunnable` (`QThreadPool`) e o uso correto do parâmetro `daemon=True` na biblioteca nativa `threading` do Python para encerramento seguro do backup ao fechar a aplicação.
- **Criação de Massa de Dados para Testes**: Foi solicitado à IA que formatasse listas de filmes clássicos respeitando a estrutura de 8 colunas separadas por ponto e vírgula, agilizando a criação dos arquivos `.csv` e `.txt` utilizados para testar a importação em lote.
- **Depuração de Erro Silencioso (AttributeError)**: Após removermos as ações redundantes da `QToolBar` na refatoração final, o botão de importação em lote parou de responder sem exibir caixas de erro na tela. Foi utilizada a IA para analisar o rastreamento do terminal, identificando rapidamente uma linha remanescente (`self.acao_importar.setEnabled(False)`) que tentava acessar a ação recém-excluída.

4. Estrutura, Execução e Considerações Finais

4.1. Estrutura do Código

A divisão do projeto foi mantida em quatro partes principais ao longo de todo o desenvolvimento: a pasta `componentes/` abriga as lógicas do diálogo modal de cadastro (`dialogo_cadastro.py`) e da janela secundária de detalhes (`janela_detalhes.py`); a pasta `dados/` guarda o repositório (`repositorio.py`) responsável pelo armazenamento local em `dados.json`, exportação/importação CSV/TXT e geração automática do diretório `backups/`; a pasta `janelas/` contém a janela principal do sistema (`janela_principal.py`) junto às classes de sinais e *workers* das threads; e o arquivo `main.py` atua como o ponto de partida da aplicação.

4.2. Instruções de Execução

Para iniciar a interface de forma correta, o usuário precisa garantir a instalação da linguagem Python na máquina. No sistema Linux ou macOS, o primeiro passo na primeira execução é criar um ambiente virtual digitando `python3 -m venv venv` no terminal. Em seguida, ativa-se o ambiente usando `source venv/bin/activate`. Com o ambiente ativo, instala-se a biblioteca gráfica executando `pip install pyside6` e inicia-se o sistema digitando `python3 main.py`.

Para o sistema Windows, o usuário pode criar e ativar o ambiente virtual no terminal com `python -m venv venv` e `venv\Scripts\activate`, instalar a dependência com `pip install pyside6` e rodar o projeto digitando o comando `python main.py`.

4.3. Considerações Finais

O projeto atingiu todos os objetivos propostos ao longo das duas etapas do trabalho. Na primeira fase foi construída uma base para a aplicação, contemplando: widgets, layouts, sinais, slots, eventos, barras de menu/ferramentas e múltiplas janela. Isso permitiu que o programa evoluísse de maneira limpa na etapa final para incorporar persistência de dados em arquivos (JSON, CSV e TXT) e execução concorrente com múltiplas threads. A organização modular em pastas separadas manteve as funcionalidades bem divididas, garantindo uma interface gráfica fluida, livre de congelamentos e que consolidou os conceitos da disciplina.

Referências

[1] THE QT COMPANY. *Qt for Python Documentation (PySide6)*. Oslo: The Qt Company, 2026. Disponível em: `https://doc.qt.io/qtforpython-6/`.

[2] FITZPATRICK, Martin. *PySide6 Tutorials: Create GUI Applications with Python & Qt6*. [S.l.]: Python GUIs, 2026. Disponível em: `https://www.pythonguis.com/`.

[3] EVANDRO, Professor. *Notas de Aula e Materiais da Disciplina de Programação Orientada a Objetos II (Aulas 01 a 08)*. Picos, PI: Universidade Federal do Piauí, 2026.

[4] SOCIEDADE BRASILEIRA DE COMPUTAÇÃO (SBC). *Modelos para Publicação de Artigos*. Porto Alegre, RS: SBC, 2026. Disponível em: `https://www.sbc.org.br/`.